

MODULE 06

子 Agent、Agent Teams、后台会话与通信

四种并发原语、隔离边界、任务账本、协调器与跨会话消息

Claude Code 的多 agent 不是一个功能，而是四种不同生命周期：同会话 subagent、协调型 team、独立 background session、跨 session messaging。选错原语会造成上下文爆炸或写冲突。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

1. 四种并发原语
2. AgentTool 生命周期
3. SkillTool 与 AgentTool 的根本差异
4. Task 系统是共享账本
5. Coordinator mode
6. Background sessions 与 agent view
7. Cross-session messaging
8. 隔离选择
9. 失败模式与治理
10. 选择决策

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 四种并发原语

原语	上下文	进程/会话	通信	适用
Subagent	隔离窗口，受父 session 管理	同会话任务	最终/进度回父	搜索、审查、局部研究
Agent Team	每个 teammate 独立	lead + teammates	共享 task + 直接消息	可分解的协作任务
Background session	完整独立会话	daemon/worker	attach/log/通知	长任务、并行项目
Cross-session	各自完整会话	本机或 cloud session	inbox/SendMessage	松耦合协作

官方明确：subagent 只能在单个 session 内回报；agent teams 的 teammate 可相互通信且用户可直接交互；background agents 是多独立 session 的运行与监控；cross-session messaging 连接本机、subagent 和 Remote Control 可见的 cloud session。 [O12, O13, O14, O15]

2. AgentTool 生命周期



- 选择 agent definition：name/description/model/tools/disallowedTools/permissionMode/memory 等。
- 创建隔离对话与 parent tool_use 关联；默认共享文件系统，不共享完整对话。
- 子 agent 独立执行 loop；工具权限既受 agent allowlist，也受父 session 可用能力约束。
- 进度以 task_started/progress/notification 或带 parent_tool_use_id 的消息回传。
- 最终结果变成父 Agent tool_result；父模型决定继续、合并或重试。 [O12, S4, R2]

SDK v0.2.141 的 `forward_subagent_text` 默认关闭：只转发子 agent 的 tool_use/tool_result 作为 heartbeat；开启后才把 text/thinking 也转发。这样默认避免把整个 sidechain 塞入主 UI/消费方。 [S4, S5]

3. SkillTool 与 AgentTool 的根本差异

维度	SkillTool	AgentTool
上下文	当前窗口注入指令	新窗口
成本	低	高，单独模型调用
隔离	无	对话隔离，可限制工具/模型
适合	确定 workflow、领域规则	大规模搜索、独立验证
返回	指令继续留在当前上下文	摘要/结果回父

历史 v2.1.88 研究估计 Agent 团队可显著放大 token；具体倍数依任务和模型变化，不能当固定常数。真正优化方法是让子 agent 返回高密度证据与结论，而不是把全文历史搬回主窗口。 [R1]

4. Task 系统是共享账本

TaskCreate/Get/List/Update 把工作分解从自然语言提升为结构化状态：owner、status、dependencies、description、result/error、timestamps。agent team 通过共享 task list 避免重复领取；CLI/SDK 通过 task lifecycle frames 展示进度。 [O13, S4, R2]

状态/事件	含义	父会活动作
task_started	后台执行开始	登记 inflight，保持 stdin
task_progress	用量/最后工具/描述	更新 UI，不终止 turn
task_notification	完成/失败摘要	唤醒 follow-up
task_updated terminal patch	补丁式终态	也必须结清 ledger
result with inflight	当前 turn 结束	不能关闭通道

5. Coordinator mode

协调器把主 agent 从执行者变成调度者：先拆任务和依赖，选择 worker，控制写权限，收集证据，解决冲突，最后合成。历史源码中 coordinator/swarm 是独立子系统；当前官方 Agent Teams 用 lead/teammates 呈现相同思想。 [O13, R2]

- 只有足够独立的子任务才并行；共享核心文件的任务应串行或分 worktree。
- worker prompt 包含目标、输入边界、禁止事项、交付格式和验收标准。
- lead 不应重新做每个 worker 的全部工作，而是校验抽样与跨结果一致性。
- 冲突优先按证据处理，不能按 agent 数量投票。
- 写 agent 与审查 agent 分离，可减少确认偏差。

示例 | 推荐的协调结构

```
Task A: API contract inventory [read-only]
Task B: implementation change [worktree B]
Task C: tests and failure injection [worktree C]
Lead: merge evidence → review diff → run acceptance suite
```

6. Background sessions 与 agent view

claude agents 提供多会话面板；可 dispatch、peek、attach、查看 blocked/done 状态。后台 supervisor 管理 worker 与 socket，claude daemon status/stop 用于诊断和恢复。每个 background session 是完整会话，能保留自身 settings、MCP、plugins 和 cwd。 [O14, O20]

- 适合持续数十分钟的测试、迁移、索引或监控；不把主终端锁死。
- 独立 session 不自动共享主会话刚刚说过的上下文，prompt 必须自足。
- agent view 中的自动命名是表层 metadata，不能作为稳定 session key。
- 后台 session 的工作区若被删除，runtime 应保留绝对路径/安全降级；v2.1.236 修复了相关回归。 [O1, O14]

7. Cross-session messaging

每个可联系 session 绑定 inbox socket；`/list-agents` 枚举 subagents、本机 sessions，以及 Remote Control 连接时可见的 cloud sessions。`SendMessage` 由模型根据 name/description 选择目标。v2.1.236 增加 `notify_when_idle`：目标下次空闲时发一次通知，无需轮询。 [O1, O15]

消息类型	建议语义
request	明确任务、输入、验收与截止条件
status	只报告状态变化，不刷屏
result	结论 + 证据 + 未解决风险
blocker	需要的人类/权限/依赖决策
idle notification	一次性唤醒，避免 sleep polling

安全边界

来自另一个 session 的消息应视为外部输入，不自动拥有用户授权。消息可以提供信息或建议，不能隐式扩大当前 session 的权限。

8. 隔离选择

隔离	文件系统	优点	代价
in-process	共享	快、启动低	写冲突、权限共享复杂
git worktree	独立工作树	适合并行写代码	合并/依赖管理
separate process	可同/异 cwd	故障隔离	进程与资源治理
remote/cloud	远端 snapshot	可扩展、离线	环境漂移、凭据与同步

规则：只读研究可共享工作区；任何同时写相同路径的 agent 应用 worktree 或串行；远端动作必须有独立身份与最小凭据；最终集成由一个 owner 执行。

9. 失败模式与治理

- 重复任务：缺共享 task ledger 或 owner 原子领取。
- 静默死亡：只等最终 result，没有 heartbeat/task progress。
- 上下文泄漏：把所有子 agent transcript 回注主窗口。
- 写冲突：多个 agent 共用 working tree，最后写覆盖先写。
- 权限漂移：恢复 teammate 时继承了旧高权限；应重新判定。
- 永不结束：后台 monitor 被错误计入必须等待的 terminal task；只追踪能可靠终止的 task type。
- 关闭过早：主 turn result 到达即关 stdin，导致后台 hook/MCP 失效。 [S2]

10. 选择决策

问题	选择
一次性局部研究、结果回主会话？	Subagent
多个角色共享任务并互相沟通？	Agent Team
多个长期独立任务集中监控？	Background sessions + agent view
已有会话之间偶发协作？	Cross-session messaging
并行改代码？	每任务 worktree + 单一集成 owner

参考资料与证据等级

[O12] 官方 | [Subagents](#)

<https://code.claude.com/docs/en/sub-agents>

[O13] 官方 | [Agent teams](#)

<https://code.claude.com/docs/en/agent-teams>

[O14] 官方 | [Agent view / background sessions](#)

<https://code.claude.com/docs/en/agent-view>

[O15] 官方 | [Cross-session messaging](#)

<https://code.claude.com/docs/en/cross-session-messaging>

[O21] 官方 | [Remote Control](#)

<https://code.claude.com/docs/en/remote-control>

[S2] 开源源码 | [Internal Query control protocol](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/query.py

[S4] 开源源码 | [Message parser](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/message_parser.py

[S6] 开源源码 | [Session reader](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/sessions.py

[R1] 社区研究 | [Dive into Claude Code v2.1.88](#)

<https://github.com/VILA-Lab/Dive-into-Claude-Code>

[R2] 社区研究 | [Claude Code Research - source analysis](#)

<https://github.com/cablate/claude-code-research>

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。